



Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

# Citation Count	40000
🕒 Difficulty	Advanced
🏢 Institution	Microsoft
≡ Keyword	Swin Transformer, Shifted Window, Hierarchical Feature Map, Vision Transformer, Self-Attention
🔗 Link	2103.14030
≡ Research Area	Vision Transformer / Computer Vision Architecture
# Year	2021

1. Introduction

이 논문은 비전 태스크 전반에서 범용 백본(**general-purpose backbone**)으로 쓸 수 있는 새로운 Vision Transformer, Swin Transformer를 제안합니다.



"Transformer를 언어가 아닌 비전에 쓰려면, 어떤 구조적 문제를 해결해야 할까?"

저자들은 기존 ViT가 단일 해상도의 feature map을 생성하고 이미지 크기에 대해 이차 복잡도를 가진다는 두 가지 핵심 한계를 지적하며, **Shifted Window 기반의 계층적 Transformer**로 이를 해결합니다.

논문이 다루는 분야

Swin Transformer는 **Computer Vision**의 다음 태스크들을 대상으로 합니다.

- **Image Classification:** ImageNet-1K top-1 정확도
- **Object Detection:** COCO box AP, mask AP
- **Semantic Segmentation:** ADE20K mIoU

즉 classification만을 위한 모델이 아니라, **dense prediction task**까지 아우르는 범용 비전 백본을 목표로 합니다.

해당 task에서 기존 연구 한계점

기존 Vision Transformer(ViT)는 단일 해상도 feature map과 이미지 크기에 대한 이차 계산 복잡도 때문에 **dense prediction task**에 적합하지 않다.

1) ViT의 고정 해상도 문제

기존 ViT는 입력 이미지를 고정된 크기의 패치로 나누고, 이 패치들을 토큰으로 취급해 전역 self-attention을 수행합니다. 그 결과 **feature map이 단일 저해상도**로만 생성됩니다.

하지만 object detection, semantic segmentation 같은 dense prediction task는 다음이 필요합니다:

- **다중 스케일 feature map** (서로 다른 크기의 객체를 다루기 위해)
- **고해상도 feature** (픽셀 수준 예측을 위해)

ViT는 이 두 가지를 모두 제공하지 못합니다.

2) Self-Attention의 이차 복잡도

전역 self-attention은 모든 토큰 쌍 사이의 관계를 계산하므로, 이미지 해상도에 대해 $O(n^2)$ 복잡도를 가집니다.

고해상도 이미지에서는 토큰 수가 폭발적으로 증가하기 때문에, ViT의 전역 attention은 **실용적이지 않습니다**.

3) 시각적 요소의 스케일 다양성

자연어에서 단어는 대체로 고정된 의미 단위이지만, 이미지에서 시각적 요소는 **크기가 매우 다양합니다**. 같은 장면에서 작은 새와 큰 자동차가 함께 있을 수 있고, 이를 같은 크기의 패치로 처리하면 정보 손실이 생깁니다.

논문의 contributions

1) Hierarchical Feature Map 생성

Swin Transformer는 CNN과 유사하게 **계층적 feature map**을 생성합니다.

- Stage 1 → Stage 4로 갈수록 해상도는 낮아지고 채널 수는 늘어남
- 기존 FPN, U-Net 등 dense prediction 기법을 그대로 활용 가능
- 다양한 스케일의 시각적 요소를 효과적으로 표현

2) Shifted Window Self-Attention으로 선형 복잡도 달성

이미지를 겹치지 않는 로컬 윈도우로 분할하고 **각 윈도우 내부에서만 self-attention**을 수행합니다.

- 윈도우 크기 M 이 고정되면 이미지 크기에 대해 $O(n)$ **선형 복잡도** 달성
- **Shifted Window** 방식으로 인접 윈도우 간 정보 교환 가능

3) 다양한 vision task에서 SOTA 달성

- ImageNet-1K 분류: **87.3% top-1 accuracy**
- COCO 객체 탐지: **58.7 box AP, 51.1 mask AP**
- ADE20K 시맨틱 세그멘테이션: **53.5 mIoU**

2. Related Work

저자들은 기존 연구를 크게 세 흐름으로 정리합니다.

CNN 계열

AlexNet → VGG → ResNet → DenseNet → EfficientNet으로 이어지는 CNN 발전 흐름을 설명합니다. CNN은 지역적 receptive field와 계층적 feature 추출이라는 inductive bias를 가지며, 이것이 실제로 시각 정보 처리에 효과적임을 인정합니다.

Self-Attention 기반 백본

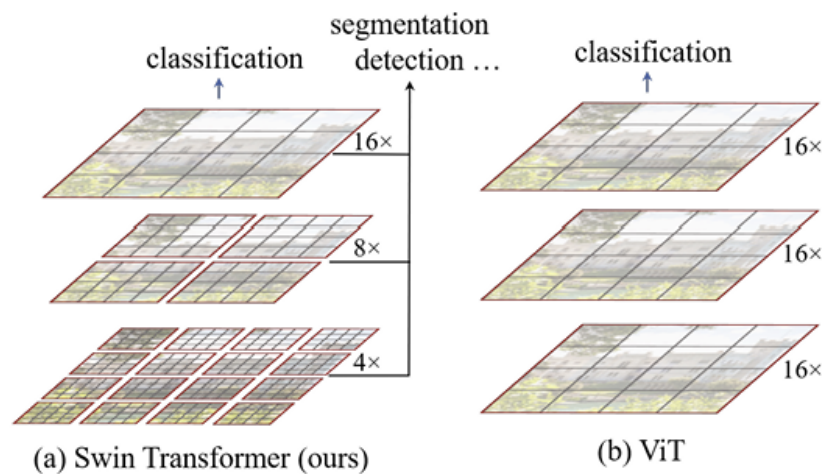
기존에도 ResNet의 일부 레이어를 local self-attention으로 교체하려는 시도가 있었지만, 슬라이딩 윈도우 방식은 하드웨어에서의 메모리 접근 비효율 때문에 실제 latency가 높았습니다. Swin Transformer는 이를 shifted window 방식으로 해결합니다.

Vision Transformer 계열

ViT는 이미지 분류에서 인상적인 결과를 보였지만 대용량 데이터(JFT-300M)가 필요했고, DeiT는 더 작은 데이터셋에서도 동작하도록 개선했습니다. 그러나 두 모델 모두 단일 해상도 feature map과 이차 복잡도 문제는 그대로 남아 있습니다.

ViT vs Swin Transformer 핵심 비교

[Figure 1 — (a) Swin Transformer vs (b) ViT 구조 비교 다이어그램]



항목	ViT	Swin Transformer
Feature Map	단일 저해상도	계층적 (4x, 8x, 16x, 32x)
Self-Attention 범위	전역 (Global)	로컬 윈도우 (Local Window)
계산 복잡도	$O(n^2)$ — 이차	$O(n)$ — 선형
윈도우 간 연결	항상 연결됨	Shifted Window로 선택적 연결
Dense task 적합성	낮음 (별도 deconv 필요)	높음 (FPN 등과 직접 호환)
위치 인코딩	절대 위치 임베딩	상대 위치 편향 (Relative position bias)
주요 활용	이미지 분류	분류 + 탐지 + 분할 (범용 백본)

제안 방법의 차별성



Swin Transformer는 CNN의 계층적 구조와 Transformer의 attention 기반 전역 모델링을 결합하여, 범용 비전 백본으로서의 두 가지 요건(다중 스케일 표현 + 효율적 계산)을 모두 충족합니다.

3. 제안 방법론

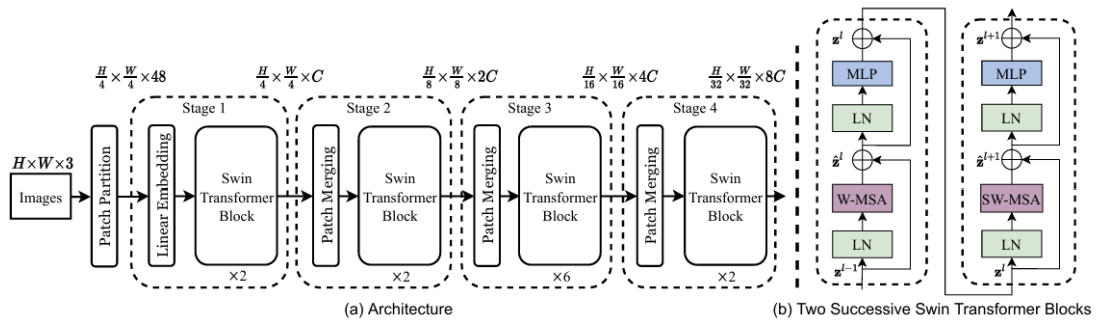
"We propose a hierarchical Transformer whose representation is computed with Shifted windows."

Main Idea

이미지를 로컬 윈도우로 나누어 self-attention을 수행하고, 레이어마다 윈도우를 이동시켜 윈도우 간 연결을 만들며, 패치를 점진적으로 합쳐 계층적 feature map을 생성한다.

전체 아키텍처 개요

[Figure 3 — Swin Transformer 전체 아키텍처 및 Swin Transformer Block 구조]



Stage	Resolution	채널 수 (Swin-T 기준)	Transformer Block 수
Stage 1	$H/4 \times W/4$	$C = 96$	2
Stage 2	$H/8 \times W/8$	$2C = 192$	2
Stage 3	$H/16 \times W/16$	$4C = 384$	6
Stage 4	$H/32 \times W/32$	$8C = 768$	2

각 Stage는 Patch Merging → Swin Transformer Block × N 구조로 이루어집니다.

[핵심 1] Patch Partition & Linear Embedding

입력 이미지를 4×4 크기의 비겹침 패치로 분할합니다.

- 각 패치의 feature 차원: $4 \times 4 \times 3 = 48$
- Linear Embedding 레이어로 임의의 차원 C로 투영

즉 입력 이미지 $H \times W \times 3$ 이 $(H/4) \times (W/4) \times C$ 토큰 맵으로 변환됩니다.

[핵심 2] Patch Merging

Stage가 깊어질수록 공간 해상도를 줄이는 역할을 합니다.

- 2×2 인접 패치들의 feature를 concatenate → 4C 차원
- Linear layer로 2C 차원으로 압축
- 해상도는 2× downsampling, 채널은 2× 증가

CNN의 pooling과 유사한 역할을 Transformer 구조에서 수행합니다.

[핵심 3] Window-based Self-Attention (W-MSA)

전역 self-attention 대신, 이미지를 크기 $M \times M$ (기본 $M=7$) 의 비겹침 로컬 윈도우로 나누고 각 윈도우 내부에서만 self-attention을 수행합니다.

계산 복잡도 비교:

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C$$

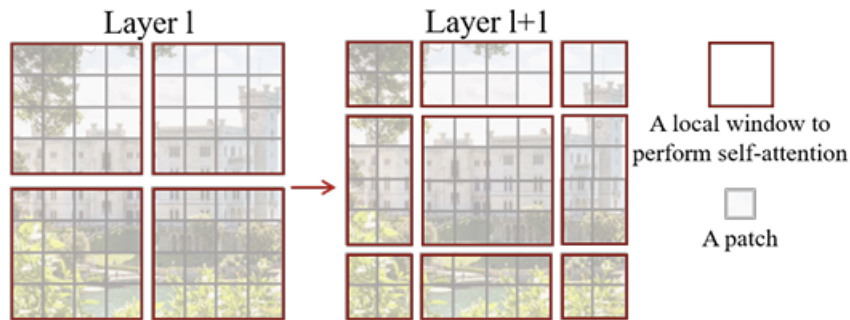
$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$$

방법	복잡도	특징
Global MSA (ViT)	$O(n^2)$ — 이미지 크기에 이차	모든 토큰 쌍 계산, 고해상도 불가
W-MSA (Swin)	$O(n)$ — M 고정 시 선형	윈도우 내부만 계산, 효율적

M 이 고정되면 이미지 크기 hw 에 대해 **선형 복잡도**가 됩니다.

[핵심 4] Shifted Window Partitioning (SW-MSA)

[Figure 2 — Shifted Window 방식 일러스트레이션 (Layer $l \rightarrow$ Layer $l+1$)]



W-MSA만 쓰면 윈도우 간 정보 교환이 없어 **모델링 능력이 제한**됩니다.

이를 해결하기 위해, 연속된 레이어에서 윈도우 분할 위치를 $(\lfloor M/2 \rfloor, \lfloor M/2 \rfloor)$ 픽셀만큼 이동시킵니다.

이렇게 하면:

- 이전 레이어 윈도우 경계를 가로지르는 새 윈도우 생성
- 인접 윈도우 간 **cross-window connection** 형성
- 전역적 정보 흐름 가능

연속된 두 Swin Transformer Block은 다음과 같이 계산됩니다:

$$\hat{\mathbf{z}}^l = \text{W-MSA}(\text{LN}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}$$

$$\mathbf{z}^l = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^l)) + \hat{\mathbf{z}}^l$$

$$\hat{\mathbf{z}}^{l+1} = \text{SW-MSA}(\text{LN}(\mathbf{z}^l)) + \mathbf{z}^l$$

$$\mathbf{z}^{l+1} = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^{l+1})) + \hat{\mathbf{z}}^{l+1}$$

[핵심 5] Efficient Batch Computation — Cyclic Shift

[Figure 4 — Cyclic Shift를 이용한 효율적 배치 연산 일러스트레이션]

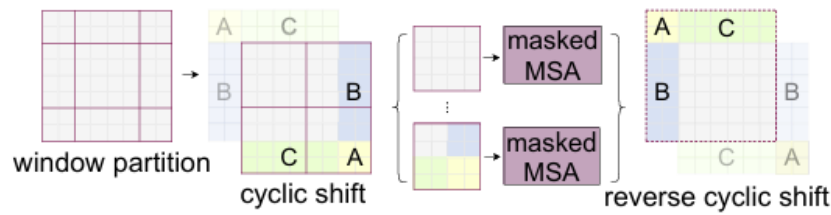


Figure 4. Illustration of an efficient batch computation approach for self-attention in shifted window partitioning.

Shifted partition은 윈도우 수를 늘릴 수 있고 ($2 \times 2 \rightarrow 3 \times 3$), 일부 윈도우가 $M \times M$ 보다 작아지는 문제가 있습니다. 단순한 패딩+마스킹 방식은 계산량이 2.25배 늘어납니다.

이를 해결하기 위해 **cyclic shift** 방식을 제안합니다:

- 이미지를 왼쪽 위 방향으로 순환 이동(cyclic-shift)
- 논리적으로 인접하지 않은 sub-window들을 **masking**으로 처리
- 윈도우 수는 regular partition과 동일하게 유지
- 결과적으로 **padding 방식 대비 13~18% 속도 향상**

[핵심 6] Relative Position Bias

Self-attention 계산 시 각 head에 **상대 위치 편향(relative position bias)** B를 추가합니다:

$$\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T / \sqrt{d} + B)V$$

- 각 축에서 상대 위치 범위: $[-M + 1, M - 1]$
- 실제로는 크기 $(2M-1) \times (2M-1)$ 의 편향 행렬 $\hat{B}$ 를 파라미터화
- 절대 위치 임베딩 대비 성능 향상 확인
- 다른 윈도우 크기로 fine-tuning 시 **bi-cubic interpolation**으로 초기화 가능

위치 인코딩 방식	ImageNet top-1	특징
No position	80.1%	위치 정보 없음
Absolute position	80.5%	ViT 방식, dense task에서 역효과
Relative position bias (Swin)	81.3%	모든 task에서 일관된 향상

Architecture Variants

모델	C	Layer 수	비교 대상
Swin-T	96	{2, 2, 6, 2}	ResNet-50 수준 복잡도
Swin-S	96	{2, 2, 18, 2}	ResNet-101 수준 복잡도
Swin-B	128	{2, 2, 18, 2}	ViT-B / DeiT-B 수준
Swin-L	192	{2, 2, 18, 2}	Swin-B 대비 2× 용량

모든 변형에서 윈도우 크기 $M=7$, 각 head의 query 차원 $d=32$, MLP expansion $\alpha=4$ 로 고정합니다.

4. 실험 및 결과

Dataset

세 가지 주요 벤치마크에서 실험합니다.

Task	Dataset	규모	평가 지표
Image Classification	ImageNet-1K	1.28M train / 50K val / 1000 classes	Top-1 accuracy
Object Detection	COCO 2017	118K train / 5K val / 20K test-dev	Box AP, Mask AP
Semantic Segmentation	ADE20K	20K train / 2K val / 3K test	mIoU (150 categories)

추가로 더 큰 용량 모델은 **ImageNet-22K** (14.2M 이미지, 22K 클래스)로 사전학습 후 fine-tuning합니다.

결과

1) Image Classification — ImageNet-1K

[Table 1 — ImageNet-1K 분류 결과 비교표]

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [58]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [63]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [63]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [63]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5
(b) ImageNet-22K pre-trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [38]	384 ²	388M	204.6G	-	84.4
R-152x4 [38]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.4
Swin-L	384 ²	197M	103.9G	42.1	87.3

Table 1. Comparison of different backbones on ImageNet-1K classification. Throughput is measured using the GitHub repository of [68] and a V100 GPU, following [63].

- 유사한 복잡도의 DeiT 대비 **+1.5% top-1 accuracy** 향상
- 22K 사전학습 시 Swin-B는 ViT-B 대비 **+2.4%** (속도는 유사)

2) Object Detection — COCO 2017

[Table 2(b) — Cascade Mask R-CNN으로 다양한 백본 비교]

(b) Various backbones w. Cascade Mask R-CNN									
	AP ^{box}	AP ₅₀ ^{box}	AP ₇₅ ^{box}	AP ^{mask}	AP ₅₀ ^{mask}	AP ₇₅ ^{mask}	param	FLOPs	FPS
DeiT-S [†]	48.0	67.2	51.7	41.4	64.2	44.3	80M	889G	10.4
R50	46.3	64.3	50.5	40.1	61.7	43.4	82M	739G	18.0
Swin-T	50.5	69.3	54.9	43.7	66.6	47.1	86M	745G	15.3
X101-32	48.1	66.5	52.4	41.6	63.9	45.2	101M	819G	12.8
Swin-S	51.8	70.4	56.3	44.7	67.9	48.5	107M	838G	12.0
X101-64	48.3	66.4	52.3	41.7	64.0	45.1	140M	972G	10.4
Swin-B	51.9	70.9	56.5	45.0	68.4	48.7	145M	982G	11.6

- 유사한 크기의 ResNe(X)t 대비 **+3.4~4.2 box AP** 일관된 향상
- System-level SOTA: **+2.7 box AP** (Copy-paste 대비), **+2.6 mask AP** (DetectoRS 대비)

3) Semantic Segmentation — ADE20K

[Table 3 — ADE20K 시맨틱 세그멘테이션 결과]

ADE20K		val	test			
Method	Backbone	mIoU	score	#param.	FLOPs	FPS
DANet [23]	ResNet-101	45.2	-	69M	1119G	15.2
DLab.v3+ [11]	ResNet-101	44.1	-	63M	1021G	16.0
ACNet [24]	ResNet-101	45.9	38.5	-	-	-
DNL [71]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [73]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [69]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [73]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [11]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [11]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [81]	T-Large [‡]	50.3	61.7	308M	-	-
UperNet	DeiT-S [†]	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B [‡]	51.6	-	121M	1841G	8.7
UperNet	Swin-L [‡]	53.5	62.8	234M	3230G	6.2

Table 3. Results of semantic segmentation on the ADE20K val and test set. [†] indicates additional deconvolution layers are used to produce hierarchical feature maps. [‡] indicates that the model is pre-trained on ImageNet-22K.

- 비슷한 계산량에서 DeiT-S 대비 **+5.3 mIoU**
- 이전 SOTA(SETR) 대비 **+3.2 mIoU**, 파라미터는 오히려 더 작음

Ablation Study

[Table 4 — Shifted Window와 Position Embedding ablation 결과]

	ImageNet		COCO		ADE20k mIoU
	top-1	top-5	AP ^{box}	AP ^{mask}	
w/o shifting	80.2	95.1	47.7	41.5	43.3
shifted windows	81.3	95.6	50.5	43.7	46.1
no pos.	80.1	94.9	49.2	42.6	43.8
abs. pos.	80.5	95.2	49.0	42.4	43.2
abs.+rel. pos.	81.3	95.6	50.2	43.4	44.0
rel. pos. w/o app.	79.3	94.7	48.2	41.9	44.1
rel. pos.	81.3	95.6	50.5	43.7	46.1

Table 4. Ablation study on the *shifted windows* approach and different position embedding methods on three benchmarks, using the Swin-T architecture. w/o shifting: all self-attention modules adopt regular window partitioning, without *shifting*; abs. pos.: absolute position embedding term of ViT; rel. pos.: the default settings with an additional relative position bias term (see Eq. (4)); app.: the first scaled dot-product term in Eq. (4).

Shifted Window의 효과:

설정	ImageNet top-1	COCO box AP	ADE20K mIoU
w/o shifting (regular window만)	80.2%	47.7	43.3
Shifted Window (Swin)	81.3% (+1.1)	50.5 (+2.8)	46.1 (+2.8)

Shifted Window는 세 가지 task 모두에서 일관된 성능 향상을 보여줍니다. 특히 object detection(+2.8 box AP)과 segmentation(+2.8 mIoU)에서 효과가 뚜렷합니다.

Speed 비교:

[Table 5 — 다양한 self-attention 방법의 실제 속도 비교]

method	MSA in a stage (ms)				Arch. (FPS)		
	S1	S2	S3	S4	T	S	B
sliding window (naive)	122.5	38.3	12.1	7.6	183	109	77
sliding window (kernel)	7.6	4.7	2.7	1.8	488	283	187
Performer [14]	4.8	2.8	1.8	1.5	638	370	241
window (w/o shifting)	2.8	1.7	1.2	0.9	770	444	280
shifted window (padding)	3.3	2.3	1.9	2.2	670	371	236
shifted window (cyclic)	3.0	1.9	1.3	1.0	755	437	278

Table 5. Real speed of different self-attention computation methods and implementations on a V100 GPU.

방법	FPS (Swin-T)	특징
Sliding window (naive)	183	레이턴시 매우 높음
Window w/o shifting	770	빠르지만 cross-window 연결 없음
Shifted window (padding)	670	비효율적 패딩 방식
Shifted window (cyclic)	755	padding 대비 13% 빠름

5. 결론

1) 좋았던 점

1. **직관적인 설계**: Shifted Window라는 아이디어가 단순하면서도 효과적
2. **세 가지 task에 걸친 종합적 검증**: 분류, 탐지, 분할 모두에서 일관된 성능 향상 입증
3. **Ablation study의 충실함**: 각 설계 요소(shifted window, position bias, cyclic shift)의 기여를 개별적으로 검증
4. **CNN과의 연속성**: FPN, UperNet 등 기존 dense prediction 파이프라인과 그대로 호환

2) 아쉬웠던 점

1. **attention map 시각화 부족**: 각 head나 레이어에서 어떤 패턴을 학습하는지에 대한 분석이 없음
2. **윈도우 크기 선택의 근거**: $M=7$ 이 최적인 이유에 대한 심층 분석이 부족
3. **Relative position bias 설계**: 왜 absolute보다 relative가 더 좋은지 이론적 설명이 약함

3) 배운점

이 논문을 통해 배운 가장 중요한 점은,



좋은 아키텍처 설계는 **문제의 특성(비전의 스케일 다양성, 고해상도)을 정확히 파악하고**, 이를 구조적 **inductive bias로 반영**하는 데서 나온다는 것입니다. Swin Transformer의 계층적 구조와 로컬 윈도우는 시각 정보의 본질적 특성에서 출발한 것입니다.

Swin Transformer는 이후 **MAE, Swin-V2, VideoSwin** 등 다양한 후속 연구의 기반이 되었으며, vision-language 통합 모델에도 광범위하게 활용됩니다.



GAN: Generative Adversarial Nets

# Citation Count	75000
🕒 Difficulty	Intermediate
🏛️ Institution	Academic
≡ Keyword	GAN, Generative Adversarial Network, Generator, Discriminator, Minimax Game
🔗 Link	1406.2661
≡ Research Area	Generative Model
# Year	2014

1. Introduction

이 논문은 생성 모델(generative model)을 학습하는 새로운 프레임워크, **Generative Adversarial Nets(GAN)**을 제안합니다.

? "딥러닝에서 판별 모델(discriminative model)은 놀라운 성공을 거뒀는데, 왜 생성 모델(generative model)은 그렇지 못했을까? 그리고 어떻게 하면 더 잘 만들 수 있을까?"

저자들은 이에 "두 모델을 서로 경쟁시키면 된다"고 답합니다.

즉, 새로운 생성 모델 구조를 제안한 것이 아니라,

기존의 **likelihood 기반 학습 패러다임을 adversarial(적대적) 게임 패러다임으로 바꾼 전환점**이 된 논문입니다.

논문이 다루는 분야

이 논문은 머신러닝의 생성 모델링(generative modeling) 분야를 다룹니다.

생성 모델이란 데이터의 분포 $p_{\text{data}}(x)$ 를 학습하여 새로운 샘플을 만들어내는 모델입니다. 대표적인 응용으로는 다음과 같은 것들이 있습니다.

- **이미지 생성**: 존재하지 않는 얼굴, 풍경, 사물 이미지 합성
- **데이터 증강**: 학습 데이터가 부족할 때 합성 데이터로 보완
- **표현 학습**: 데이터의 잠재 구조를 비지도 방식으로 학습
- **조건부 생성**: 텍스트 설명에 맞는 이미지 생성 등

Introduction에서는 당시 딥러닝의 가장 눈부신 성과가 **판별 모델(discriminative model)**, 즉 고차원 입력을 클래스 레이블로 매핑하는 모델에서 나왔다고 설명합니다. Backpropagation, dropout, piecewise linear unit 덕분에 이런 판별 모델들이 잘 작동했습니다.

반면 **생성 모델(deep generative model)**은 상대적으로 덜 주목받았는데, 그 이유를 저자들은 두 가지로 정리합니다.

해당 task에서 기존 연구 한계점

기존 생성 모델은 최대 우도 추정(maximum likelihood estimation) 과정에서 다루기 어려운 확률 계산(intractable probabilistic computation)이 발생하거나, 생성 문맥에서 piecewise linear unit의 이점을 활용하기 어렵다.

1) Intractable 확률 계산

기존의 생성 모델들, 예를 들어 **RBM(Restricted Boltzmann Machine)**, **DBM(Deep Boltzmann Machine)**, **DBN(Deep Belief Network)** 등은 학습 과정에서 다음과 같은 문제를 겪었습니다.

- **partition function** (정규화 상수 Z)의 계산이 불가능하거나 매우 어려움
- 이를 근사하려면 **MCMC(Markov Chain Monte Carlo)** 방법이 필요한데, mixing 문제가 발생
- **Variational inference** 같은 근사 추론도 별도의 복잡한 설계가 필요

즉, 모델이 "좋다"는 것을 학습 신호로 삼기가 근본적으로 어렵습니다.

- 이전까지: **likelihood**를 최대화하려 하지만, **likelihood 자체를 계산하기 어려움**
- 이 논문 이후: **likelihood**를 직접 계산하지 않고, **판별자(D)의 실수를 통해 간접적으로 학습**

2) Piecewise Linear Unit 활용의 어려움

ReLU 같은 piecewise linear unit은 gradient가 잘 흐르기 때문에 판별 모델에서 큰 성공을 거뒀습니다. 그러나 기존 생성 모델들은 **피드백 루프(feedback loop)** 구조를 가지기 때문에, 활성화 함수가 unbounded activation을 일으켜 이런 unit을 사용하기 어려웠습니다.

반면 GAN에서 Generator는 단방향(forward propagation)으로만 샘플을 생성하기 때문에, **ReLU 등 piecewise linear unit을 자유롭게 활용**할 수 있습니다.

논문의 contributions

1) Adversarial Framework: 두 모델의 경쟁으로 생성 학습

이 논문의 가장 큰 contribution은 **Generator G와 Discriminator D를 동시에 학습시키는 adversarial 프레임워크**를 제안한 것입니다.

저자들은 이를 **위조지폐범과 경찰**의 비유로 설명합니다.



위조지폐범(G): 가능한 한 진짜 같은 위조 지폐를 만들려 한다.

경찰(D): 진짜 지폐와 위조 지폐를 구별하려 한다.

경쟁이 계속되면 결국 위조 지폐는 진짜와 구별할 수 없게 된다.

공식적으로는 다음의 **minimax 게임**으로 표현됩니다.

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

- $D(x)$ 는 입력 x 가 실제 데이터일 확률
- $G(z)$ 는 노이즈 z 를 실제 데이터처럼 보이는 샘플로 변환
- D 는 이 값을 **최대화**하려 하고 (진짜/가짜 잘 구별)

- G는 이 값을 **최소화**하려 한다 (D를 속이려 함)

이 contribution이 중요한 이유는, 당시 생성 모델들이 모두 **likelihood**를 명시적으로 다루야 했던 것과 달리, GAN은 **likelihood**를 전혀 계산하지 않고도 생성 분포를 학습할 수 있다는 점을 처음으로 보였기 때문입니다.

2) Markov Chain 없는 순수 Backpropagation 기반 학습

GAN의 두 번째 핵심 contribution은 **Markov chain**이나 근사 추론(**approximate inference**)이 전혀 필요 없다는 점입니다.

- G와 D 모두 **multilayer perceptron(MLP)**으로 구성
- 학습은 **backpropagation + dropout**만으로 가능
- 샘플 생성은 **forward propagation**만으로 가능

이는 당시 생성 모델들이 반드시 요구했던 MCMC, variational inference 등의 복잡한 절차와 극명하게 대비됩니다.

모델 유형	학습 방식	샘플링	p(x) 계산
Deep directed (DBN 등)	Inference 필요	문제 없음	AIS로 근사
Deep undirected (RBM 등)	MCMC로 partition function 근사	Markov chain 필요	AIS로 근사
Generative autoencoder	Mixing-power 트레이드오프	Markov chain 필요	Parzen 근사
GAN (이 논문)	D와 G 동기화만 필요	문제 없음 (forward pass)	Parzen 근사

즉, GAN은 계산 복잡도 관점에서 이전 생성 모델들보다 훨씬 단순하고 효율적입니다.

3) 이론적 수렴 보장

세 번째 contribution은 보다 이론적인 차원으로, 저자들은 **GAN의 global optimum이 존재하고, 그것이 $p_g = p_{data}$ 임을 수학적으로 증명**합니다.

- **Proposition 1:** 주어진 G에 대해 최적의 D는 $D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$
- **Theorem 1:** minimax 게임의 global minimum은 $p_g = p_{data}$ 일 때 달성되며, 이때 $C(G) = -\log 4$
- 이 최솟값은 **Jensen-Shannon Divergence**와 연결됨

$$C(G) = -\log(4) + 2 \cdot \text{JSD}(p_{data} \| p_g)$$

JSD는 두 분포가 같을 때만 0이 되므로, $p_g = p_{data}$ 일 때 global minimum이 달성됩니다.

이 수학적 근거가 중요한 이유는, 단순한 경험적 프레임워크가 아니라 **이론적으로 올바른 학습 목표**를 갖는다는 것을 보여주기 때문입니다.

4) 다양한 확장 가능성 제시

논문은 Conclusion에서 GAN 프레임워크가 다음과 같이 확장될 수 있음을 예고합니다.

- **Conditional GAN:** c를 G와 D 모두에 입력으로 주면 $p(x|c)$ 학습 가능
- **Semi-supervised learning:** D의 feature를 분류기에 활용
- **Learned approximate inference:** z를 예측하는 보조 네트워크 학습

이 방향들은 이후 CGAN, InfoGAN, CycleGAN, StyleGAN 등 수많은 후속 연구의 씨앗이 됩니다.